

Spring Boot OOP Concepts — Real Project Understanding

Why Beginners Get Confused

Most beginners learn:

- syntax
- definitions
- examples

But real projects use OOP concepts together.

The important question is:

Why is this concept used in real projects?

1. Class & Object

Meaning

- Class = Blueprint/template
 - Object = Real instance created from class
-

Real Project Example

E-commerce app:

- User
- Product
- Order
- Payment

These are represented using classes.

Example:

```
public class User {  
  
    String name;  
    String email;  
    int age;  
  
}
```

Creating object:

```
User user1 = new User();  
  
user1.name = "Mayur";  
user1.email = "mayur@gmail.com";  
user1.age = 22;
```

Where Used in Spring Boot

Used everywhere:

- Entity classes
- DTOs
- Request classes
- Response classes
- Service classes
- Repository classes

Spring Boot API Example

```
@PostMapping("/save")  
public String saveUser(@RequestBody User user)
```

Here:

- User = class
- user = object

Spring automatically converts JSON into object.

How To Know When To Create a Class

Ask:

Is this a real thing/data in my application?

If yes → create class.

Examples:

- User
 - Product
 - Cart
 - Payment
 - Student
 - Employee
-

2. Constructor

Meaning

Constructor initializes object values automatically.

Without Constructor

```
User u = new User();  
u.name = "Mayur";  
u.email = "abc@gmail.com";
```

Too much repetitive code.

With Constructor

```
public class User {  
  
    String name;  
    String email;
```

```
User(String name, String email){  
    this.name = name;  
    this.email = email;  
}  
}
```

Usage:

```
User u = new User("Mayur", "abc@gmail.com");
```

Constructor Injection in Spring Boot

```
@Service  
public class UserService {  
  
    private final UserRepository repo;  
  
    public UserService(UserRepository repo){  
        this.repo = repo;  
    }  
}
```

Spring automatically provides repository object.

Real Meaning

UserService cannot work without UserRepository.

Constructor ensures:

- dependency is compulsory
- object becomes ready immediately
- cleaner code
- safer code

3. Initialization

Actual Meaning

Initialization means:

Preparing object with required values/dependencies before using it.

Real-Life Analogy

Before riding bike:

- petrol needed
- key needed
- engine ready

That preparation = initialization.

Example Without Initialization

```
class User {  
    String name;  
    String email;  
}
```

```
User u = new User();  
  
System.out.println(u.name);
```

Output:

```
null
```

Object is incomplete.

Example With Initialization

```
class User {  
  
    String name;  
    String email;  
  
    User(String name, String email){  
        this.name = name;  
        this.email = email;  
    }  
}
```

Usage:

```
User u = new User("Mayur", "mayur@gmail.com");
```

Now object is ready immediately.

Initialization in Spring Boot

Dependency Initialization

```
@Service  
public class UserService {  
  
    private final UserRepository repo;  
  
    public UserService(UserRepository repo){  
        this.repo = repo;  
    }  
}
```

UserService needs UserRepository to work.

Spring initializes dependency automatically.

Where Initialization Is Used

1. Database Connection

```
DataSource dataSource
```

Spring initializes database connection.

2. Service Dependencies

```
OrderService(PaymentService paymentService)
```

OrderService needs payment service.

3. Required User Data

```
User("Mayur", "mayur@gmail.com")
```

Object initialized with required information.

4. Configuration Values

```
JwtService(secretKey)
```

JWT service needs secret key.

5. Collections

Correct:

```
List<String> names = new ArrayList<>();
```

Wrong:

```
List<String> names;  
names.add("A");
```

Error:

NullPointerException

Because list was not initialized.

Why Initialization Is Important

Initialization prevents:

- null values
- crashes
- invalid state
- incomplete objects

How To Know Initialization Is Required

Ask:

Can this object work properly without this value?

If NO → initialize it.

Examples:

Object	Required Initialization
User	name, email
OrderService	PaymentService
JwtService	secret key
DatabaseService	DB connection
ArrayList	memory allocation

4. Inheritance

Meaning

Reuse common properties and methods.

Real Project Scenario

Different users:

- Admin
- Customer
- Seller

All have:

- name
- email

Instead of repeating code:

```
public class User {  
  
    String name;  
    String email;  
}
```

Child class:

```
public class Admin extends User {  
  
    String adminRole;  
}
```

Where Used in Spring Boot

Common places:

- BaseEntity

- Exception classes
 - Security users
 - Common response classes
-

Example

```
@Entity
public class BaseEntity {

    @Id
    Long id;

    LocalDateTime createdAt;
}
```

```
@Entity
public class Product extends BaseEntity {

    String name;
}
```

Now Product automatically gets:

- id
 - createdAt
-

How To Know When To Use Inheritance

Ask:

Is this object a type of another object?

Examples:

- Admin IS A User
 - Dog IS A Animal
 - Bike IS A Vehicle
-

5. Polymorphism

Meaning

Same method behaves differently.

Real Project Scenario

Payment system:

- UPI
- Credit Card
- PayPal

All process payments differently.

Interface

```
public interface PaymentService {  
  
    void pay();  
}
```

UPI implementation:

```
public class UpiPayment implements PaymentService {  
  
    public void pay(){  
        System.out.println("UPI Payment");  
    }  
}
```

Card implementation:

```
public class CardPayment implements PaymentService {  
  
    public void pay(){  
        System.out.println("Card Payment");  
    }  
}
```

```
}  
}
```

Usage:

```
PaymentService payment = new UpiPayment();  
payment.pay();
```

Later:

```
payment = new CardPayment();
```

Same method:

```
pay()
```

Different behavior.

That is polymorphism.

Where Used in Spring Boot

Very heavily used:

- Services
 - Interfaces
 - Repositories
 - Authentication
 - Notification systems
 - Payment systems
-

Real Spring Boot Structure

```
Controller  
↓  
Service  
↓  
Repository
```

↓
Database

OOP Concepts Used in Layers

Layer	OOP Concept
Entity	Class/Object
Service Injection	Constructor
BaseEntity	Inheritance
Payment/Notification	Polymorphism

One-Line Summary

Concept	Purpose
Class/Object	Represent real-world data
Constructor	Initialize required data/dependencies
Inheritance	Reuse common code
Polymorphism	Same action, different behavior

Beginner Mindset Shift

Do not ask only:

- What is inheritance?
- What is constructor?

Instead ask:

- What problem exists?
- Why normal code becomes messy?
- How OOP solves it?

That is real project thinking.